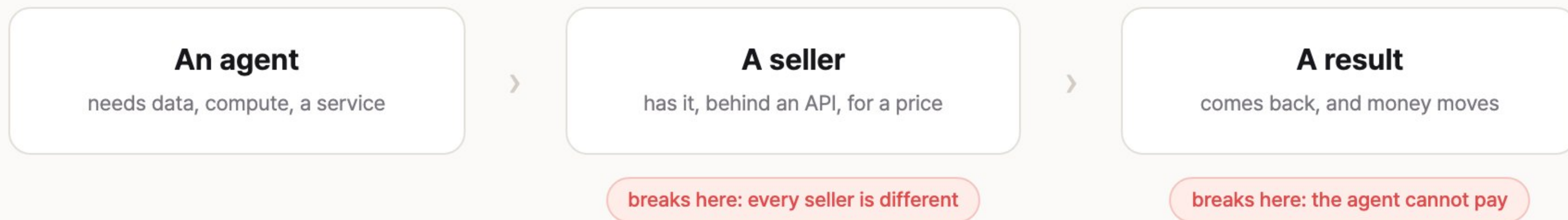


x402, the Agent Capability Market, and **h402**.

YoungHwi Cho (Project7) | Co-founder at HUNT



Agent commerce is an agent buying something on its own, and today the purchase breaks in two places.



A human buyer signs up, learns the product, and pays. An agent can do the middle part. The first and last parts still need a person, and that is the whole gap this talk is about.

The gap splits into two problems: learning every seller, and paying each one.

problem one

Does the agent have to learn every seller?

Every API has its own docs, SDK, request shape and error style. Teaching an agent all of them, one at a time, does not scale.

problem two

How does the agent actually pay?

An agent cannot open an account, accept terms, or clear a captcha. Until this year there was no answer that did not route through a person.

Giving one agent four real APIs by hand means four accounts, four keys, four SDKs and four test cycles.

Nansen

smart money flows

- request API access
- pick a paid tier
- store the key
- learn their schema
- handle rate limits
- test, then retest

from \$99 / month

Exa

web search

- create an account
- add a card
- store the key
- install the SDK
- map the response
- test, then retest

usage billed monthly

Google Maps

place details

- GCP project + billing
- enable the API
- restrict the key
- learn the field masks
- watch the quota
- test, then retest

billing account required

ElevenLabs

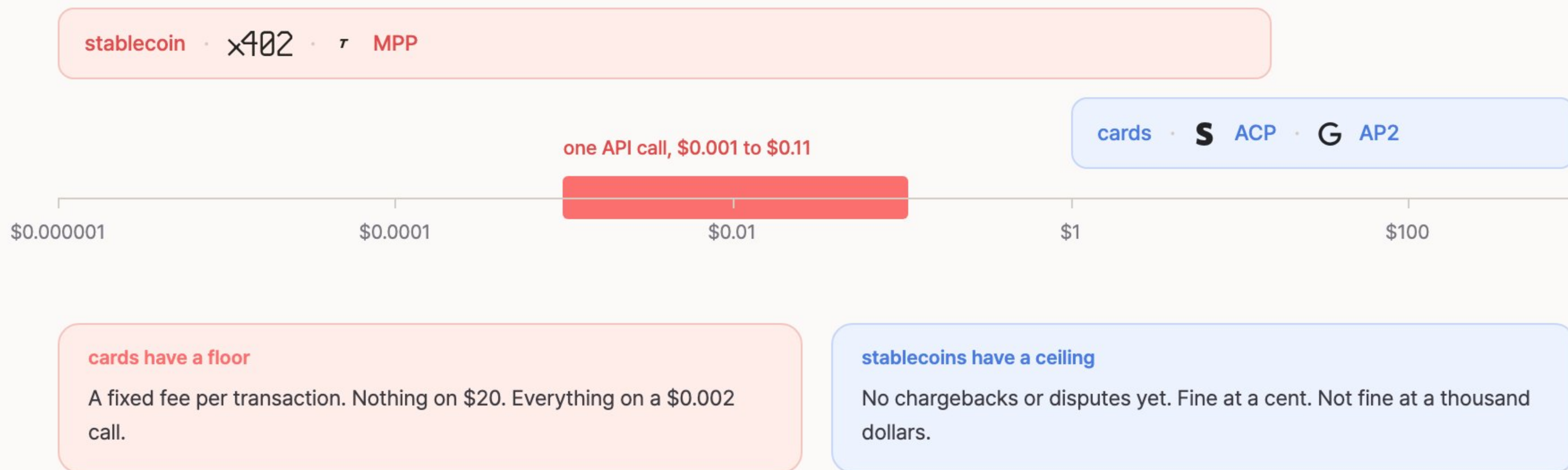
text to speech

- sign up
- choose a plan
- store the key
- pick a voice id
- handle audio streams
- test, then retest

character quota per plan

Four vendors, four billing relationships, four request shapes. And a human in every one of the first three steps.

API calls are moving to stablecoins, led by x402. Shop checkouts stay on cards, led by Stripe and Google.



x402 puts the price in an HTTP 402 response and lets the client pay inside the retry.

```
$ GET /routes/nansen/crypto/smart-money-flows
HTTP/1.1 402 Payment Required { "amount": "52500", "asset": "USDC", "network": "eip155:8453" }
$ GET /routes/nansen/crypto/smart-money-flows X-PAYMENT: <signed EIP-3009 authorization>
HTTP/1.1 200 OK { "data": [ ... ] }
```

1

Ask

The seller answers 402 with the exact price.

2

Sign

The agent signs an EIP-3009 authorization offchain. No gas.

3

Settle

A facilitator verifies it and moves the USDC onchain.

4

Deliver

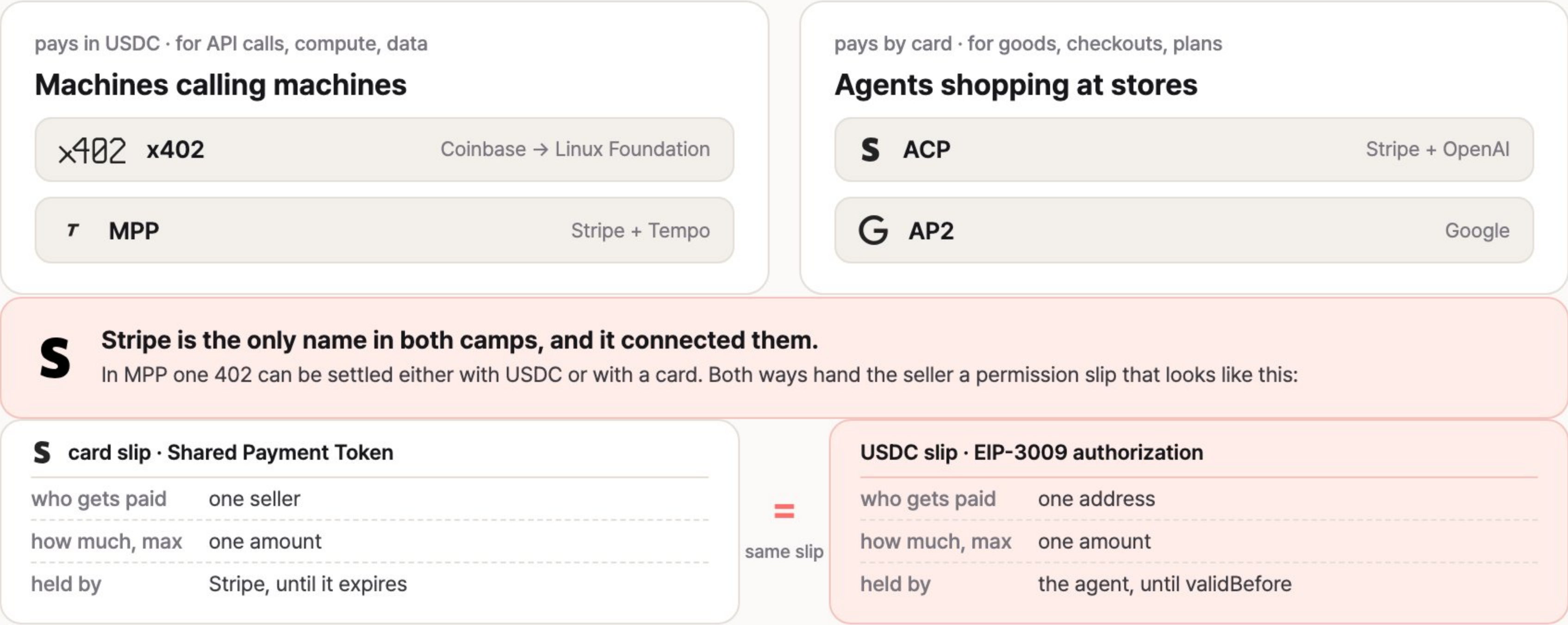
The result comes back with a payment reference.

30.47M transactions **\$1.36M** volume **20.25k** buyers **17k** sellers past 30 days

1997 402 reserved in HTTP, unused **2025** Coinbase publishes x402 **Apr 2026** moved to the Linux Foundation

The request shown is a real one against h402-test.hunt.town, amount in micro USDC, fee inclusive. Ecosystem figures from [x402scan](#), past 30 days, read Sep 2026.

Two camps: USDC rails built for machines calling machines, and card rails built for agents shopping at existing stores.





**A capability market for agents.
Mount it once, call any listed
API, **pay per call.****

```
/routes/{provider}/{category}/{action}
```



The same four APIs on h402: one call path each, no accounts, and the price shown before the call.

by hand, per vendor

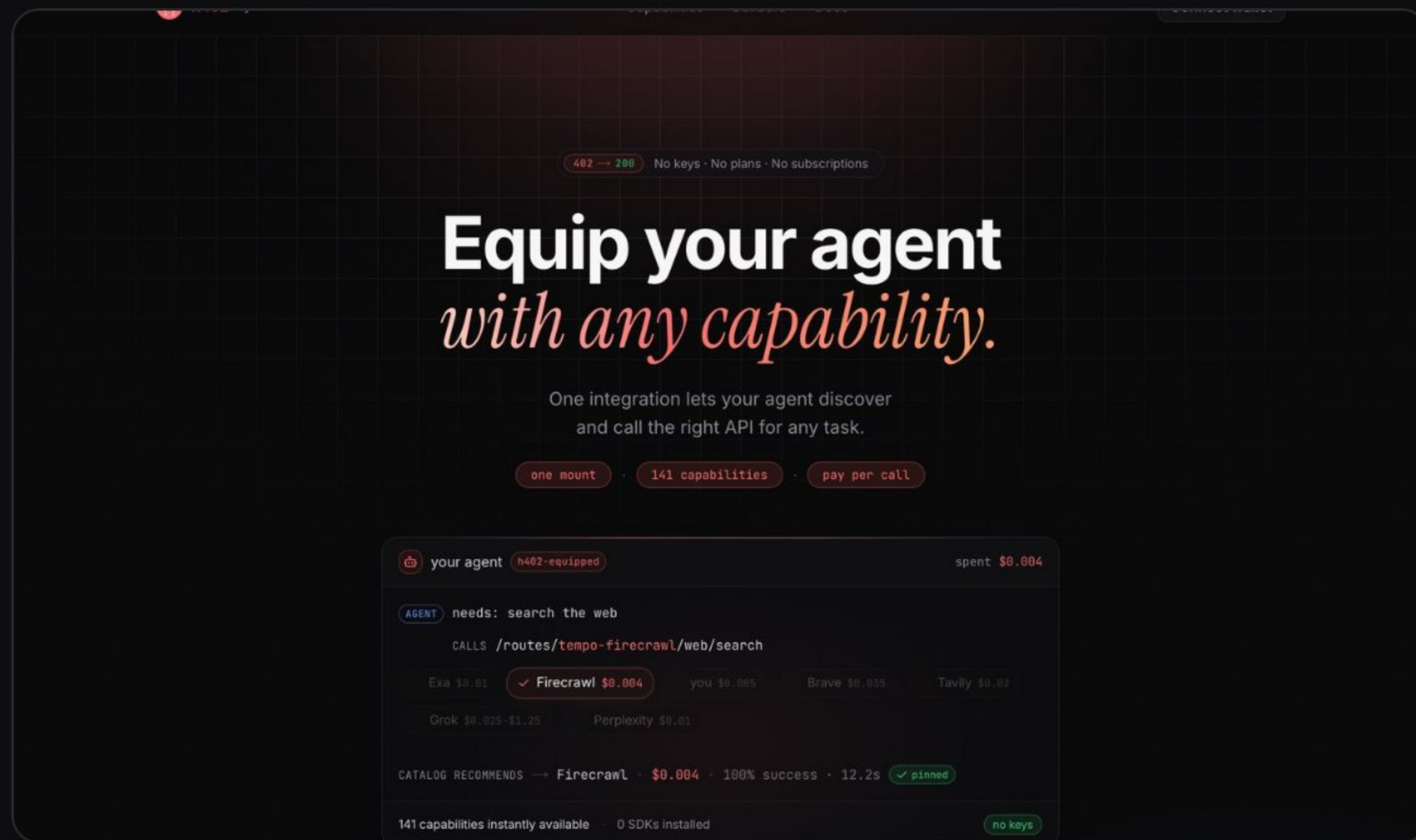
Nansen	account · tier · key · schema · limits · tests
Exa	account · card · key · SDK · mapping · tests
Google Maps	GCP · billing · key · masks · quota · tests
ElevenLabs	account · plan · key · voice · streams · tests

on h402, one line each

/routes/nansen/crypto/smart-money-flows	\$0.05
/routes/stableenrich-exa/web/search	\$0.004
/routes/stableenrich-googlemaps/maps/place-details	\$0.02
/routes/blockrun-elevenlabs/ai/text-to-speech	from \$0.001

Four vendors became four lines. The agent never met any of them. Prices are live quotes and move.

What is live today: 141 capabilities, 71 provider contracts, every enabled route paid-tested.



141

capabilities

across 16 categories

71

provider contracts

competing on shared task paths

100%

paid-tested

every enabled route, with real calls

Base

USDC over x402

testing on mainnet, launch in Q3

Where h402 sits: the agent pays by itself, and it gets a verified pick instead of a list.

the agent gets
a verified pick

- **h402**
paid-tested, ranked, one provider pinned

the agent gets
a list

- **RapidAPI · Apify**
one account, many APIs
- **MCP gateways**
one endpoint, many tools

- **Circle agent marketplace**
84 services, no quality signal
- **Alsa · other resellers**
wrappers, flat prices

a human sets up payment

the agent pays by itself

h402 runs on both rails: buyers always pay over x402, and providers are paid over x402 unless MPP is all they have.

Buyer → h402

every call

x402 v2 exact, USDC on Base

The agent signs an EIP-3009 authorization. The Coinbase CDP facilitator settles it into the h402 treasury, fee-free.

```
PAYMENT-REQUIRED → sign → PAYMENT-SIGNATURE → settled
```

h402 → provider

most of them

x402 exact, USDC on Base

The canonical provider rail, paid from the h402 operating wallet.

```
select.ts: x402-first // even if the provider also speaks MPP
```

h402 → provider

MPP-only

MPP one-shot charge on Tempo

Only when a provider has no x402 rail at all. Signed via mppx; anything but a 6-decimal USDC charge fails closed.

```
tempo/charge // the only MPP shape we settle
```

x402-first

A dual-rail provider settles over x402, always.

one-shot, exact

No sessions, no streams, no open tabs.

the agent sees none of it

One call, one price, one result.

Every call path pins one recommended provider. The pin is earned from paid calls, not from a listing.

01

before listing

We buy it

No route enters the catalog until we have paid for it ourselves and seen a real response.

`100%` of enabled routes paid-tested

02

on every call

We measure it

Success rate, p50 latency, and price, from live paid traffic. Buyers' calls count too.

`score = quality-adjusted price`

03

same task

We rank it

Providers on the same call path are scored against each other, not against their own docs.

`needs ≥ 5` recent calls to be trusted

04

the result

We pin one

The highest score becomes the default the agent receives. A provider that fails twice in 15 minutes is dropped from the default.

`defaultProvider ← top score, health-checked`

h402 is a two-sided market: agents on one side, API providers on the other, and the score routes the traffic.

agent side

Mount once, pay per call

- no accounts, no keys, no SDK per vendor
- one call path per capability
- price shown before every call
- gets the pinned provider, not a menu

h402 score

success · latency · price

from real paid calls

top score gets pinned

provider side

List once, get paid per call

- no checkout, no billing, no key issuance to build
- paid in USDC over x402, or MPP if that is all you have
- compete on the same path as everyone else
- a better score wins the pin, and the pin is the traffic

On a directory, the first listing wins. Here the best-performing provider wins, and it is re-decided on every call. If you run a good API, this is the platform where quality converts to revenue most directly.

71

provider contracts live on h402

17k

sellers already on x402, past 30 days

20.25k

buyers on x402, past 30 days

5 of ours

already selling on x402scan:
StableEnrich, glim.sh, twit.sh, Otto AI,
BlockRun

A fresh session mounts h402 and pays for **real data**.

Nothing installed. Watch what it decides to buy, and what it shows before it spends.

What can you do with h402? Show the available capabilities, providers, and prices.

Worked for 1m 8s >

The live staging catalog currently has **147 capabilities** across 16 categories. Prices below are catalog prices in USDC (1,000,000 micro-USDC = \$1); a paid request's final x402 quote includes h402's 5% buyer fee.

Category	Routes	What it covers	Catalog price range
Crypto	41	prices, DEX, RPC, wallets, smart money, perps	Free–\$0.05
Finance	15	stocks, FX, SEC, macro, research	Free–\$0.50
Social	14	X, TikTok, Instagram, Reddit	Free–\$0.06
AI	14	chat, image/video/music generation, TTS, PDF/OCR	Free–\$5.00
Web	10	search, answers, scrape, sitemap, screenshots,	\$0.001–\$1.25

Three things to take home, depending on what you build.

Agent commerce splits into two problems. The first is mostly solved. The second is where the standards are being written right now, and where h402 sits.

depending on what you build

01 Building agents?

Your next design problem is a budget and a spend cap, not a prompt. Nobody has good defaults yet.

02 Selling an API?

Agents are customers who cannot sign up for anything. A middleware and a day is the whole lift.

03 Picking a protocol?

Do not. They are not competing for the same money. Sit behind an abstraction and let the price decide.